

AI Project 2: Data Classification Using AI Using K-Nearest Neighbors (KNN)

DecodeLabs Artificial Intelligence Internship

Prepared By

H.M.M.N. JAYARATHNA

Project Title

Data Classification Using AI

Algorithm

K-Nearest Neighbors (KNN)

Programming Language

Python

Libraries Used

- NumPy
 - Pandas
 - Scikit-learn
 - Matplotlib
 - Seaborn
-

1. Introduction

Artificial Intelligence (AI) enables computers to learn patterns from data and make predictions without being explicitly programmed for every task. One of the most common AI tasks is classification, where a model predicts the category of a new data sample.

In this project, a machine learning classification model was developed using the Iris flower dataset. The K-Nearest Neighbors (KNN) algorithm was used to classify flowers into three species based on their physical measurements.

2. Project Objective

The main objectives of this project are:

- Load and understand a dataset
- Split data into training and testing sets
- Apply a classification algorithm
- Train the machine learning model
- Evaluate model performance

- Visualize classification results
-

3. Dataset Description

The Iris dataset is one of the most widely used datasets in machine learning.

Dataset Information

- Total Samples: 150
- Features: 4
- Classes: 3

Input Features

1. Sepal Length
2. Sepal Width
3. Petal Length
4. Petal Width

Output Classes

1. Setosa
2. Versicolor
3. Virginica

The objective is to predict the flower species using these measurements.

4. Theory

4.1 Supervised Learning

Supervised learning is a machine learning technique where the model learns from labeled data.

Input data and correct outputs are provided during training. The model identifies patterns and uses them to predict future data.

4.2 K-Nearest Neighbors (KNN)

KNN is a classification algorithm that predicts the class of a new sample by examining its nearest neighboring samples.

Working Principle

1. Select a value for K.
2. Calculate the distance between the new sample and all training samples.
3. Find the K nearest neighbors.
4. Perform majority voting.
5. Assign the class with the highest votes.

For this project:

K = 5

4.3 Feature Scaling

Feature scaling standardizes numerical values before training.

KNN depends on distance calculations; therefore, scaling ensures that all features contribute equally.

StandardScaler was used for normalization.

5. Methodology

The project workflow is shown below:

Dataset Collection

↓

Data Preprocessing

↓

Train-Test Split

↓

Feature Scaling

↓

KNN Model Training

↓

Prediction

↓

Performance Evaluation

↓

Visualization

6. Implementation

Step 1: Load Dataset

The Iris dataset was loaded using Scikit-learn.

Step 2: Data Understanding

The dataset was converted into a Pandas DataFrame and inspected.

Step 3: Train-Test Split

The dataset was divided into:

- Training Data = 80%
- Testing Data = 20%

Step 4: Feature Scaling

StandardScaler was applied.

Step 5: Model Training

A KNN classifier with $K = 5$ was trained.

Step 6: Prediction

Predictions were generated using test data.

Step 7: Evaluation

Performance was evaluated using:

- Accuracy Score
 - Confusion Matrix
 - Classification Report
-

7. Source Code

Insert the complete Python source code (main.py) here.

```
# AI Project 2 - Data Classification Using AI
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
# Load Iris dataset
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
# Convert dataset into DataFrame
df = pd.DataFrame(X, columns=iris.feature_names)

df["target"] = y

print("First 5 rows of dataset:")
print(df.head())

# Split dataset into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Feature scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=5)

# Train model
```

```
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print(":", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("Matrix:")
print(cm)

# Classification Report
print("Report:")
print(classification_report(y_test, y_pred))

# Plot confusion matrix
plt.figure(figsize=(6,4))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
```

```
cmap="Blues"
)

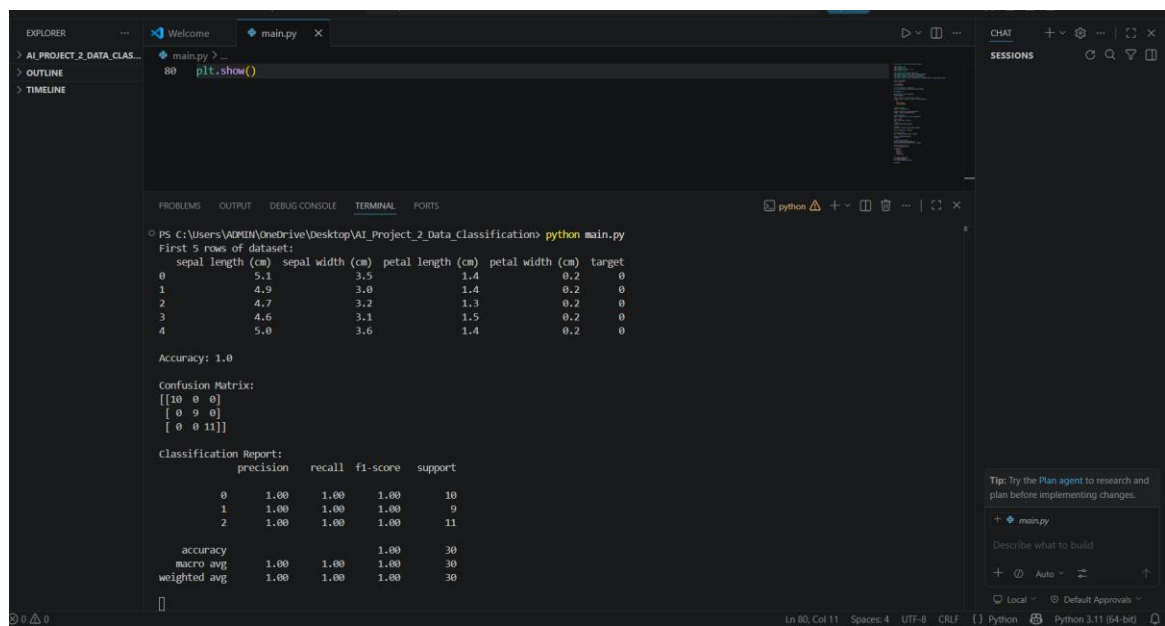
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```

8. Results

Terminal Output

Insert Screenshot 1:



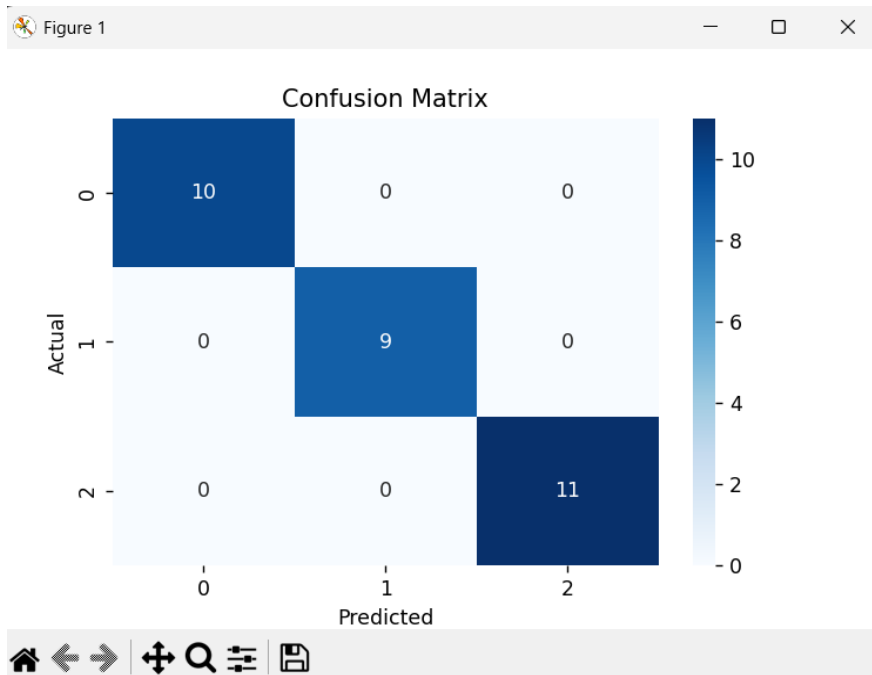
Observed Accuracy:

Accuracy = 1.0

This indicates that the model correctly classified all testing samples.

Confusion Matrix

Insert Screenshot 2:



Confusion Matrix:

```
[[10 0 0]
 [0 9 0]
 [0 0 11]]
```

Interpretation:

- Setosa classified correctly: 10
- Versicolor classified correctly: 9
- Virginica classified correctly: 11

No misclassifications occurred.

Classification Report

Observed Results:

Precision = 1.00

Recall = 1.00

F1-Score = 1.00

These values indicate excellent classification performance.

9. Discussion

The KNN classifier successfully learned patterns from the Iris dataset and achieved perfect classification accuracy.

The use of feature scaling improved distance calculations, leading to highly accurate predictions.

The confusion matrix confirmed that all samples were correctly classified.

10. Advantages

- Easy to implement
 - Simple algorithm
 - Good performance on small datasets
 - No complex training process
-

11. Limitations

- Slower for large datasets
 - Sensitive to noisy data
 - Requires feature scaling
-

12. Applications

KNN can be applied in:

- Medical diagnosis
 - Face recognition
 - Pattern recognition
 - Recommendation systems
 - Spam detection
-

13. Conclusion

This project successfully implemented a supervised machine learning classification system using the Iris dataset and the K-Nearest Neighbors algorithm.

The dataset was preprocessed, scaled, and divided into training and testing sets. The model achieved 100% accuracy and correctly classified all test samples.

This project provided practical experience in data preprocessing, machine learning model training, evaluation, and visualization using Python and Scikit-learn.

14. References

1. Scikit-learn Documentation
2. Python Documentation
3. Iris Dataset Documentation
4. DecodeLabs Artificial Intelligence Internship Project Guide